

Common Lisp 用カラー文字列表示用パッケージ「print-color-string」使用方法

ひとつのファイル「print-color-string.lisp」に関連する関数群が収められ、文字列をカラーで画面に出力する関数「print-colored-string」を使えるようにするパッケージ(ライブラリ)です。

```
(print-colored-string 'blue "Blue-string.")
```

とすると「blue」と定義された色で文字列「Blue string.」が出力されます。後述の方法で色合は調整できます。

文字属性を追加して出力する

キーワード引数:bold :italic :underline :invert :strike に[*nil*]以外を与えて呼び出すと

:bold t	太字または高輝度にする。
:italic t	斜体にする。
:underline t	下線をつける
:invert t	前景色と背景色を入れ替える。
:strike t	打ち消し線を引く。

という属性付きで表示できます。同時に複数の属性を指定できます。

```
* (print-colored-string 'blue "Blue-string.")
Blue-string.
"Blue-string."
* (print-colored-string 'blue "Blue-string." :bold t)
Blue-string.
"Blue-string."
* (print-colored-string 'blue "Blue-string." :bold t :italic t)
Blue-string.
"Blue-string."
* (print-colored-string 'blue "Blue-string." :bold t :italic t :underline t)
Blue-string.
"Blue-string."
* (print-colored-string 'blue "Blue-string." :bold t :italic t :underline t :invert t)
Blue-string.
"Blue-string."
* (print-colored-string 'blue "Blue-string." :bold t :italic t :underline t :invert t :strike t)
Blue-string.
"Blue-string."
*
```

使用できる色は ANSI 互換端末で実行した場合は基本 8 色で

Black
Red

Green
Yellow
Blue
Magenta
Cyan
White

の記号名で指定できます。現在では大半が xterm 互換端末(256 色モード)だと思いますが、その場合は

Black
Red
Green
Yellow
Blue
Magenta
Cyan
Gray

という記号名で指定できます。24 bit TrueColor 端末の場合も 256 色モードで動作します。

xterm 互換端末では記号名の色に対する色合いを変更可能

xterm 互換端末の場合は各色に定義されているカラー・コードを変更して好みの色合いに変えることができます。ANSI 互換端末では変更できません。実行中の端末が ANSI 互換端末なのか xterm 互換端末なのかは print-color-string パッケージ起動時に「PATH」環境変数の値を読み取って自動的に決定しています。ほぼないと思いますが、一切の文字属性修飾機能を持たない「dumb(ダム)」端末かどうかも判定しています。dumb 端末だった場合は一切の文字修飾情報の出力を行いません。

```
(show-basic-colors)
```

と実行すると端末情報に基づいて定義された上記のそれぞれの基本色(8 色)がカラー・コードとともに表示されます。

```
(show-all-color)
```

とすると端末情報に基づいて ANSI 互換端末であれば全 8 色が、xterm 互換端末であれば 256 色すべての色見本がカラー・コードとともに表示されます。例えば red のカラー・コードは初期値として 124 と定義していますが、これをカラー・コード 201 の色に変更したい場合は

```
(adjust-xterm-color-code 'red 201)
```

とします。変更前の色&カラー・コードと変更後の色&カラー・コードが表示されます。

```
(show-basic-colors)
```

を再度実行すると red のカラー・コードが 196 から 201 に変わり、色も変わっていることが確認できます。

```

* (adjust-xterm-color-code 'red 201)
現在の red のカラー・コードは 196: 0123456789 0123456789
新しい red のカラー・コードは 201: 0123456789 0123456789
T
* (show-basic-colors)
      xterm current colors
symbol code  前景色  反転色
red      201: 0123456789 0123456789
green    46:  0123456789 0123456789
blue     21:  0123456789 0123456789
cyan     51:  0123456789 0123456789
magenta  201: 0123456789 0123456789
yellow   226: 0123456789 0123456789
gray     239: 0123456789 0123456789
black    232: 0123456789 0123456789

see also (show-all-color)
T
* █

```

以後

```
(print-colored-string 'red "Red string.")
```

で表示される red はカラー・コード 201 の色になります。

表示する色見本の出力範囲を制御する

256 色モードの場合、すべての色を表示すると 1 画面に収まりません。画面に表示する色見本の出力範囲と種類(グラデーション色)を指示することができます。出力するカラー・コードの範囲を指定するには

```
(show-xterm-color-sample :range '(1 (40 51)))
```

のようにします。これでカラー・コードの 1 と 40..51 の範囲のカラー・コードだけを表示します。指定可能な引数のパターンと意味は、この関数を引数なしで実行すると表示されます。

```
(show-xterm-color-sample)
```

xterm 互換端末モードでの

```
(show-color-sample)
```

は xterm 互換端末で利用できる 256 色すべてを表示する

```
(show-xterm-color-sample :range '(0 255))
```

と同じです。

```
* (show-xterm-color-sample :range '(1 (40 51)))
  xterm 256 colors
code  前景色      背景色
  1: 0123456789 0123456789
  xterm 256 colors
code  前景色      背景色
 40: 0123456789 0123456789
 41: 0123456789 0123456789
 42: 0123456789 0123456789
 43: 0123456789 0123456789
 44: 0123456789 0123456789
 45: 0123456789 0123456789
 46: 0123456789 0123456789
 47: 0123456789 0123456789
 48: 0123456789 0123456789
 49: 0123456789 0123456789
 50: 0123456789 0123456789
 51: 0123456789 0123456789
NIL
* (show-xterm-color-sample)
"指定された範囲のxtermで定義された色を表示する。

(引数なし)      この関数ドキュメントを表示する。
:range nil      この関数ドキュメントを表示する。
:range 1        カラー・コード1の色を表示。
:range '(1)     カラー・コード1の色を表示。
:range '(0 15)  カラー・コード0..15の範囲の色を表示。
:range '(1 3 15) カラー・コード1, 3, 15の3色を表示。
:range '((0 15) (215 255)) カラー・コード0..15と215..255の範囲の色を表示。
:range '(1 (3 15) (215 225)) カラー・コード1とカラー・コード3..15、215..225の範囲の色を表示。

"
*
```

色のグラデーションによる違いを見たい場合は

(show-xterm-gradation :kind 'blue-gradation)

のように指定します。これで表示可能な青のグラデーションが表示されます。指定可能な引数のパターンを知りたいときは引数なしで呼び出すと呼び出し例を表示します。

```

* (show-xterm-gradation :kind 'blue-gradation)
--- R:0->0, G:0->0, B:0->5 ---
code  前景色      背景色
16: 0123456789  0123456789
17: 0123456789  0123456789
18: 0123456789  0123456789
19: 0123456789  0123456789
20: 0123456789  0123456789
21: 0123456789  0123456789
* (show-xterm-gradation)
"xterm互換端末で配色の参考とするための類型化した色のグラデーションを表示する関数。

(show-xterm-gradation)                この関数のドキュメントを表示する。
(show-xterm-gradation :kind nil)       この関数のドキュメントを表示する。
(show-xterm-gradation :kind 'red-to-green) 赤から緑へのグラデーションを表示する。
(show-xterm-gradation :kind 'green-to-blue) 緑から青へのグラデーションを表示する。
(show-xterm-gradation :kind 'blue-to-red)  青から赤へのグラデーションを表示する。
(show-xterm-gradation :kind 'red-gradation) 濃い赤から明るい赤へのグラデーションを表示する。
(show-xterm-gradation :kind 'green-gradation) 濃い緑から明るい緑へのグラデーションを表示する。
(show-xterm-gradation :kind 'blue-gradation) 濃い青から明るい青へのグラデーションを表示する。
(show-xterm-gradation :kind 'monochrome)    濃い灰色から明るい灰色へのグラデーションを表示する。
(show-xterm-gradation :kind 'mono)          濃い灰色から明るい灰色へのグラデーションを表示する。
"
*

```

複数のセッションで同じ色合いを自動設定する

Common Lisp のセッションを終了すると初期設定のカラー・コード(今の red の場合は 196)に戻ります。常に同じ変更後のカラー・コードの色で使い続けたい場合は初期設定ファイル

```
init-print-color-string.lisp
```

に

```
(adjust-xterm-color-code 'red 201)
```

と書き込んでおきます。初期設定ファイルはカレント・ディレクトリ→ホーム・ディレクトリの順に探して、見つけたら読み込んでセッション開始前に内容を実行します。

```
(adjust-xterm-color-code 'red 201 :quiet t)
```

とすると確認表示が出力されません。adjust-xterm-color-code の説明は

```
(documentation 'adjust-xterm-color-code 'function)
```

とすると表示されます。

```
* (documentation 'adjust-xterm-color-code 'function)
"xターミナルの256色から red, green, blue, cyan, magenta, yellow, black, gray の8色に
どの色（カラーコード）を与えるかを再定義するための関数。
第1引数に色名を指定し、第2引数に設定したい色のカラーコードを与える。
この関数を引数なしで呼び出すと、現在のカラーコードの見本を出力する。
第1引数を指定して第2引数を省略すると第1引数で指定した現在の色見本を表示する。
第1引数がシンボルの場合は、そのシンボルに第2引数で指定されたカラー・コードを設定し、色見本を表示する。
第1引数と第2引数を両方指定すると第1引数で指定された色を第2引数で指定されたカラー・コードの
色に変更して、変更前と変更後の色見本を表示する。
"
```

したがって、初期設定ファイル実行中に関数 `adjust-xterm-color-code` を実行して、その確認表示を行わせたくない場合は

```
(adjust-xterm-color-code 'red 201 :quiet t)
```

と初期設定ファイル(`init-print-color-string.lisp`)書き込んでおきます。

基本色の色合いを変更せずに好みの色を使う

上で述べた基本色を関数 `adjust-xterm-color-code` で変更せずに好みの色を指定することもできます。関数 `print-colored-string` の第1引数には直接カラー・コード(数値)を指定できます。例えばカラー・コード 196 の赤色を出力したい場合は

```
(print-colored-string 201 "Red string.")
```

と書くこともできます。また適当なシンボル、例えば `my-red` にカラー・コードの 201 を定義しておき、そのシンボル `my-red` を使うこともできます。

```
(setf my-red 201)
(print-colored-string my-red "Red string.")
```

xterm 互換端末か ANSI 互換端末かに従って正しい範囲のカラー・コードを使っているかのチェックを行っています。指定したカラー・コードが正しい範囲(xterm 互換端末なら 0..255、ANSI 互換端末なら 0..7)でなかった場合はモノクロで出力します。

カラー・コードを数値で指定することは可読性が低下するため推奨しません。また適当なシンボルにカラー・コードを与える方法もシンボルの値が意図しない変更を受ける可能性があります。出来る限りクオート(')付きで基本色を指定する方法をお薦めします。

グラデーション見本の表示

表示する色の参考となるように色相・明度・彩度のグラデーション表示を行う関数も実装しています。

show-all-gradation	表現可能な色相・明度・彩度のグラデーションを表示する。
show-hue-gradient	指定した2つの色の間の色相変化を表示する。
show-saturation-gradient	指定した純色から高明度グレーへの彩度変化を表示する。
Show-brightness-gradient	指定した純色の明るさ変化を表示する。

実例を示します。

```
* (show-all-gradation)
--- 端末で使用可能なグラデーション (xterm 256色) ---

*** 色相変化グラデーション ***

R -> G グラデーション (show-hue-gradient :from 'red' :to 'green')
201: 0123456789 0123456789
165: 0123456789 0123456789
129: 0123456789 0123456789
 93: 0123456789 0123456789
 57: 0123456789 0123456789
 21: 0123456789 0123456789
 27: 0123456789 0123456789
 33: 0123456789 0123456789
 39: 0123456789 0123456789
 45: 0123456789 0123456789
 51: 0123456789 0123456789
 50: 0123456789 0123456789
 49: 0123456789 0123456789
 48: 0123456789 0123456789
 47: 0123456789 0123456789
 46: 0123456789 0123456789

G -> B グラデーション (show-hue-gradient :from 'green' :to 'blue')
 46: 0123456789 0123456789
 47: 0123456789 0123456789
 48: 0123456789 0123456789
 49: 0123456789 0123456789
 50: 0123456789 0123456789
 51: 0123456789 0123456789
 45: 0123456789 0123456789
 39: 0123456789 0123456789
 33: 0123456789 0123456789
 27: 0123456789 0123456789
 21: 0123456789 0123456789

B -> R グラデーション (show-hue-gradient :from 'blue' :to 'red')
 21: 0123456789 0123456789
 57: 0123456789 0123456789
 93: 0123456789 0123456789
129: 0123456789 0123456789
---
```

```
* (show-hue-gradient :from 'green :to 'blue)
```

46:	0123456789	0123456789
47:	0123456789	0123456789
48:	0123456789	0123456789
49:	0123456789	0123456789
50:	0123456789	0123456789
51:	0123456789	0123456789
45:	0123456789	0123456789
39:	0123456789	0123456789
33:	0123456789	0123456789
27:	0123456789	0123456789
21:	0123456789	0123456789

```
NIL
```

```
* █
```

```
* (show-saturation-gradient 'red :to-light t)
```

201:	0123456789	0123456789
207:	0123456789	0123456789
213:	0123456789	0123456789
219:	0123456789	0123456789
225:	0123456789	0123456789
231:	0123456789	0123456789

```
NIL
```

```
* (show-brightness 'yellow :to-light t)
```

```
--- R:0->0, G:0->0, B:0->5 ---
```

code	前景色	背景色
16:	0123456789	0123456789
58:	0123456789	0123456789
100:	0123456789	0123456789
142:	0123456789	0123456789
184:	0123456789	0123456789
226:	0123456789	0123456789

```
* █
```


パッケージのコンパイル方法

パッケージを利用するためには予めコンパイルしておく必要があります。注意を要するのは、コンパイル時にメモリ上に構築されたシンボル名との衝突を避けるために、コンパイル済みモジュールをロードする前に、一旦処理系を終了し、再起動直後のクリーンな状態で .fasl ファイルをロードする必要があることです。あくまでも「警告」なので無視できますが、処理系によってはデバッガが介入するので面倒です。sbcl の場合だと以下の手順でコンパイルすれば警告が出ません。

[1]> sbcl	←処理系起動。
* (compile-file "print-color-string.lisp")	←開発者向け脚注あり。
* (exit)	←一旦、処理系を終了する。
[2]> sbcl	←処理系の再起動。
* (load "print-color-string.fasl")	←処理系再起動後にロードする。

あとは

```
> (print-color-string:print-colored-string 'blue "Blue string.")
```

パッケージ名プレフィックスを付けずに使いたい場合はあらかじめパッケージを use しておきます。

```
* (use-package :print-color-string)
* (print-colored-string 'blue "Blue string.")
```

と使います。以上。

開発者向け脚注

関数 `adjust-xterm-color-code` の定義で `&optional` と `&key` の両方を使用しているため、コンパイラによっては、この関数のコンパイル時に「コードの実行効率が悪い」として警告が出ます。実行が集中する使用頻度が高い関数ではないので警告を無視して構いません。精神衛生上どうしても抑止したい場合は、

```
(handler-bind ((warning #'muffle-warning))
  (compile-file "print-color-string.lisp"))
```

としてコンパイルすれば警告は出ません。■